

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division:C Batch: 2 Practical No: 7

Roll no: 19 PRN No: 12410562 Name of Student: Parth Kahane

Subject Name : Deep Learning

Problem Statement : Implement transfer learning using pre-trained AlexNet, VGG16, ResNet50, and EfficientNetB0 models for image classification, and compare their performance.

Algorithm :

Step 1: Start

Step 2: Import Required Libraries

Import TensorFlow, NumPy, Matplotlib, Pandas, and other required Keras libraries.

Step 3: Load the Dataset

Load the CIFAR-10 dataset directly using tensorflow.keras.datasets.

Step 4: Preprocess the Dataset

1. Extract training and testing images and labels.
2. Convert the class labels into categorical (one-hot encoded) format.
3. Resize the images from 32×32 pixels to 224×224 pixels.
4. Create batches of images using the TensorFlow data pipeline.

Step 5: Implement VGG16 Transfer Learning

1. Load the pre-trained VGG16 model with ImageNet weights.

2. Remove the original classification layer by setting `include_top=False`.
3. Freeze the pre-trained layers.
4. Add Global Average Pooling, Dense, Dropout, and Softmax layers.
5. Compile the model using the Adam optimizer and categorical cross-entropy loss.
6. Train the model using the training dataset.
7. Evaluate the trained model using the test dataset.

Step 6: Implement EfficientNetB0 Transfer Learning

1. Load the pre-trained EfficientNetB0 model with ImageNet weights.
2. Remove the original classification layer.
3. Freeze the pre-trained layers.
4. Add Global Average Pooling, Dense, Dropout, and Softmax layers.
5. Compile the model.
6. Train the model using the training dataset.
7. Evaluate the trained model using the test dataset.

Step 7: Compare Model Performance

1. Collect the accuracy and loss values of both models.
2. Create a comparison table.
3. Plot a bar graph showing the accuracy of VGG16 and EfficientNetB0.

Step 8: Select the Best Model

Compare the performance of both models and identify the model with higher classification accuracy.

Step 9: Stop

Code & Output:

```
[1] import tensorflow as tf
    print(tf.config.list_physical_devices('GPU'))

... [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
[2] import tensorflow as tf
    import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd

    from tensorflow.keras.datasets import cifar10
    from tensorflow.keras.utils import to_categorical
```

```
[3] (x_train, y_train), (x_test, y_test) = cifar10.load_data()

    print(x_train.shape)
    print(x_test.shape)

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 1898s 11us/step
(50000, 32, 32, 3)
(10000, 32, 32, 3)
```

```
(10000, 32, 32, 3)
```

```
[4] y_train = to_categorical(y_train, 10)
    y_test = to_categorical(y_test, 10)
```

```
[5] IMG_SIZE = 224
    BATCH_SIZE = 32

    train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
    test_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test))

    train_ds = train_ds.shuffle(10000)
    train_ds = train_ds.map(
        lambda x, y: (
            tf.image.resize(tf.cast(x, tf.float32), (IMG_SIZE, IMG_SIZE)),
            y
        ),
        num_parallel_calls=tf.data.AUTOTUNE
    )

    test_ds = test_ds.map(
        lambda x, y: (
            tf.image.resize(tf.cast(x, tf.float32), (IMG_SIZE, IMG_SIZE)),
            y
        ),
        num_parallel_calls=tf.data.AUTOTUNE
    )

    train_ds = train_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
    test_ds = test_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

```
ass_7_DL.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

[5]
train_ds = train_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
test_ds = test_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

[6]
from tensorflow.keras.applications import VGG16
from tensorflow.keras.layers import Input, Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.models import Model

vgg_base = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

vgg_base.trainable = False

inputs = Input(shape=(224, 224, 3))

x = vgg_base(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)

outputs = Dense(10, activation='softmax')(x)

vgg_model = Model(inputs, outputs)

vgg_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
ass_7_DL.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 4s 0us/step

[8]
history_vgg = vgg_model.fit(
    train_ds,
    validation_data=test_ds,
    epochs=3
)

Epoch 1/3
1563/1563 331s 212ms/step - accuracy: 0.7672 - loss: 0.6772 - val_accuracy: 0.7979 - val_loss: 0.5870
Epoch 2/3
1563/1563 330s 211ms/step - accuracy: 0.7783 - loss: 0.6400 - val_accuracy: 0.8054 - val_loss: 0.5646
Epoch 3/3
1563/1563 330s 211ms/step - accuracy: 0.7871 - loss: 0.6205 - val_accuracy: 0.8103 - val_loss: 0.5528

[9]
from tensorflow.keras.applications import EfficientNetB0

efficient_base = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

efficient_base.trainable = False

inputs = Input(shape=(224, 224, 3))

x = efficient_base(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
```

ass_7_DL.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[9] ✓ 4s

outputs = Dense(10, activation='softmax')(x)

efficient_model = Model(inputs, outputs)

efficient_model.compile(
optimizer='adam',
loss='categorical_crossentropy',
metrics=['accuracy']
)

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 — 2s 0us/step

[10] ✓ 4m

history_efficient = efficient_model.fit(
train_ds,
validation_data=test_ds,
epochs=3
)

... Epoch 1/3
1563/1563 — 121s 59ms/step - accuracy: 0.8460 - loss: 0.4597 - val_accuracy: 0.8957 - val_loss: 0.3102
Epoch 2/3
1563/1563 — 65s 41ms/step - accuracy: 0.8791 - loss: 0.3514 - val_accuracy: 0.9047 - val_loss: 0.2832
Epoch 3/3
1563/1563 — 64s 41ms/step - accuracy: 0.8899 - loss: 0.3217 - val_accuracy: 0.9051 - val_loss: 0.2813

[11] ✓ 1m

vgg_loss, vgg_accuracy = vgg_model.evaluate(test_ds)

efficient_loss, efficient_accuracy = efficient_model.evaluate(test_ds)

print("VGG16 Accuracy:", vgg_accuracy * 100)

print("EfficientNetB0 Accuracy:", efficient_accuracy * 100)

ass_7_DL.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

✓ 1m

313/313 — 56s 180ms/step - accuracy: 0.8103 - loss: 0.5528
313/313 — 11s 34ms/step - accuracy: 0.9051 - loss: 0.2813
VGG16 Accuracy: 81.02999925613403
EfficientNetB0 Accuracy: 90.50999879837036

[12] ✓ 0s

results = pd.DataFrame(
Model: ['VGG16', 'EfficientNetB0'],
Accuracy (%): [
vgg_accuracy * 100,
efficient_accuracy * 100
]
)
results

Model Accuracy (%)
0 VGG16 81.029999
1 EfficientNetB0 90.509999

Next steps: [Generate code with results](#) [New interactive sheet](#)

[] Start coding or generate with AI.